



北京師範大學
BEIJING NORMAL UNIVERSITY

《计算机图形学》课程实验

实验四：网格参数化 (参考操作教程)

授课教师：张鸿文

助教：李玉慧

课程网页：<https://zhanghongwen.cn/cg>

计算机图形学实验四——网格参数化

1. 配置 DensePose 的 python 环境

首先从云盘下载本实验所需压缩包，包括代码与数据（新版压缩包仅修改了 detectron 库的安装与使用，方便 IDE 编辑及 notebook 执行；上一堂课可成功运行的，不需要重新下载）。

打开 conda 终端，运行以下命令。命令会依次新建 DensePose 环境、从外部下载所需包、从本地 DensePose\modules 目录中安装所需的 chumpy 和 detectronba 包、进入包含 python 代码文件与 notebook 教程文件的 DensePose\notebooks 目录。

```
conda create -n DensePose python=3.12
conda activate DensePose

conda install matplotlib numpy scipy
pip install jupyter notebook
pip install cython==0.29.36 pycocotools opencv-python==4.11.0.86

cd [YOUR PROJECT ROOT]\DensePose
cd modules\chumpy
pip install .
cd ../detectron
pip install .

cd ../../notebooks
```

```
Proceed ([y]/n)? y
```

其中[YOUR PROJECT ROOT]\DensePose 中的[YOUR PROJECT ROOT]为你解压后代码所在的文件夹；cd 命令中出现的“..”表示上一级目录、“.”表示当前目录，这种表示也会在代码中处理文件或文件夹路径时用到。

如果该绝对路径在其他磁盘中，需要先进入该磁盘才能成功运行 cd 命令，如：

```
D:
cd D:\projects\DensePose
```

如果之前创建过 DensePose 环境，现在想要重新创建，可以通过 conda remove 删除旧环境。

```
conda activate base
conda remove -n DensePose --all
```

2. 复习理论课“网格参数化”一节，运行四个示例程序

在上一步中已经进入了 notebooks 文件夹，此时可以先用 ls 命令查看文件夹内的文件，方便复制 python 文件名；在 DensePose 环境中直接运行即可，此处提供多种在终端中运行的方式。

```
ls
python COCO-on-SMPL.py
python .\COCO-Visualize.py

# 在非 notebooks 文件夹下运行，或通过 IDE 运行，需根据代码提示修改绝对路径
python [YOUR PROJECT ROOT]\DensePose\notebooks\RCNN-Texture-Transfer.py

# 在非 notebooks 文件夹下运行，或通过 IDE 运行，需根据代码提示修改绝对路径
cd [YOUR PROJECT ROOT]\DensePose
python notebooks\RCNN-Visualize-Results.py

cd notebooks
```

具体内容在 notebooks 有详细介绍，此处不再赘述。核心是理解各数组的维度与次序。如 RGB 图像各通道大小为 0-255、为 uint8 格式 ($2^8=256$)；IUV 图像中 I 为 0-25，U、V 均为 0-199，也为 unit8 格式。

Ipynb 格式的教程文件可以用多种 IDE 打开查看，如 pycharm、vscode、或 jupyter 网页版等；本实验不要求修改运行 notebooks，只修改 py 文件即可。其中，如果你自行寻找教程并运行 notebooks，也可以将修改运行后的 ipynb 文件作为实验文档及代码提交；jupyter 与 notebook 包已在第一步安装到 DensePose 环境下。

3. 在示例程序“COCO-on-SMPL.py”的基础上，编写新函数，查看清晰的手部关键点

首先观察已有的全身观察方式与面部观察方式，对比函数 `smpl_view_set_axis_full_body` 与函数 `smpl_view_set_axis_face`，发现核心是 `ax.view_init`、`ax.set_xlim`...几个函数。

https://matplotlib.org/stable/api/as_gen/mpl_toolkits.mplot3d.axes3d.Axes3D.view_init.html

查阅函数文档后可知，`view_init` 的前三个参数为仰角、方位角、翻滚角，可用于控制观察方向，`ax.set_xlim`、`set_ylim`、`set_zlim` 三个函数的前两个参数分别为 x 轴、y 轴、z 轴的视图边界限制，可用于控制观察平移。

结合实验二、三中平移、旋转、摄像机的概念，调整相机视图的参数。

```
fig = plt.figure(figsize=[16,4])

def smpl_view_set_axis_face(ax, azimuth=0, elev=0):
    ## Manually set axis
    ax.view_init(elev, azimuth)
    max_range = 0.1
    ax.set_xlim( 0- max_range, 0+ max_range)
    ax.set_ylim( 0.7- max_range, 0.7+ max_range)
    ax.set_zlim( -3 - max_range, -3 + max_range)
    ax.axis('off')

# 手部观察的仰角设置旋转-90度
ax = fig.add_subplot(144, projection='3d')
```

```
ax.scatter(Z,X,Y,s=0.2,c='k')
smpl_view_set_axis_face(ax,0,-90)

plt.show()
```



4. 在示例程序“COCO-Visualize.py”的基础上，自选图像及标注，展示人体关键点与 mask

分析“COCO-Visualize.py”中的文件路径代码，我们可以得到标注的路径及示例图像路径，除了肉眼观察路径结构外，我们也可以直接 print 输出绝对路径。

```
dp_coco = COCO( os.path.join(coco_folder,
'annotations/densepose_coco_2014_minival.json'))
print(os.path.join(coco_folder,
'annotations/densepose_coco_2014_minival.json'))
...
im_name = os.path.join( coco_folder + 'val2014', im['file_name'] )
print(im_name)
```

查看“densepose_coco_2014_minival.json”，可以发现其包含了 DensePose COCO 数据集中我们所需的所有标注，不需要更多下载。

在实验压缩包中的“val2014_part.zip”中已经给出了挑选后的十张图像，解压后将我们挑选的图像放在刚刚得到的‘val2014’文件夹中，也可以从官网下载完整图像数据集，自行选择图像，需要注意检查图像序号是否在 im_ids 数组中。

以 81988 图像为例，替换后直接运行。可以看到，81988 在 im_ids 数组中，表示其带标注。

```
im_ids = dp_coco.getImgIds()
print(np.sort(im_ids))

Selected_im = 81988
print(Selected_im in im_ids)
```

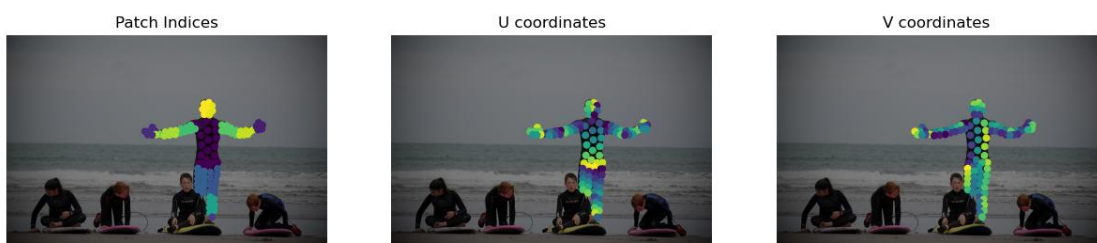
```
[ 785    872    885 ... 580197 581317 581357]
True
```



def

```
TransferTexture(TextureIm,im,IUV):
    print(TextureIm.shape, im.shape, IUV.shape)
    print("TextureIm ", TextureIm.dtype, TextureIm.min(), TextureIm.max())
    print("im ", im.dtype, im.min(), im.max())
    print("IUV ", IUV.dtype, IUV.min(), IUV.max())
```

我们已经得到了关键点的 I、U、V 数组的可视化；观察可知，标注的关键点是稀疏的。



5. 在 "COCO-Visualize.py"与"RCNN-Texture-Transfer.py"的基础上，将自选图像的纹理替换为示例纹理。

首先查看"COCO-Visualize.py"中我们已经有的数组内容

```
for ann in anns:
    .....
    print(Point_x.shape, Point_y.shape, Point_I.shape,
          Point_U.shape, Point_V.shape)
```

```

print("x", Point_x.dtype, Point_x.min(), Point_x.max())
print("y", Point_y.dtype, Point_y.min(), Point_y.max())
print("I", Point_I.dtype, Point_I.min(), Point_I.max())
print("U", Point_U.dtype, Point_U.min(), Point_U.max())
print("V", Point_V.dtype, Point_V.min(), Point_V.max())

```

```

(97,) (97,) (97,) (97,) (97,)
x float64 276.2408688096439 510.44095010196463
y float64 134.1426526069641 361.70398667279414
I float64 1.0 24.0
U float64 0.04733338579535484 0.9964996576309204
V float64 0.11278954893350601 0.8707180619239807

```

可以看到在自选图像中，关键点有 97 个。再结合 plt.scatter 的函数定义可知，x、y 数组虽然为 float64，但内容是各关键点的坐标（以像素为单位）；I 数组虽然类型为 float64，但实际内容为 1-24 的整数，而 0 表示无映射；U、V 数组为标准化后的坐标，类型为 float64，范围从 0.0-1.0。

再检查“RCNN-Texture-Transfer.py”中函数 TransferTexture 所需要的输入格式，

```

def TransferTexture(TextureIm,im,IUV):
    print(TextureIm.shape, im.shape, IUV.shape)
    print("TextureIm ", TextureIm.dtype, TextureIm.min(),
TextureIm.max())
    print("im ", im.dtype, im.min(), im.max())
    print("IUV ", IUV.dtype, IUV.min(), IUV.max())

```

```

(24, 200, 200, 3) (512, 176, 3) (512, 176, 3)
TextureIm float64 0.0 1.0
im float64 0.0 0.0
IUV uint8 0 253

```

其中纹理图集 TextureIm 大小为 24*200*200，表示模型被分割为 24 个部分，每部分对应的纹理图大小为 200*200（像素）；调用时背景图像 im 设为全黑背景，此处值全为 0.0；I、U、V 的范围为 0-255，内容为整数。此处 IUV 的大小与自选图像的大小一致，均为 512*176，而非关键点的 IUV 值。

将 TransferTexture 函数直接复制到“COCO-Visualize.py”中，修改 IUV 数组的大小与元素类型，修改自选图像的元素类型。“COCO-Visualize.py”用变量 I（指 image）表示原图像，注意将其与 IUV 中的 I 辨别。

```

# 分别初始化 IUV 数组，形状为原图像（[Height, Width, Color]）的前两维
（[Height, Width]）
I_im = np.zeros(I.shape[:-1]).astype(np.uint8)
U_im = np.zeros(I.shape[:-1]).astype(np.uint8)
V_im = np.zeros(I.shape[:-1]).astype(np.uint8)
print(I.shape, I_im.shape)

## For each ann, scatter plot the collected points.
for ann in anns:
    bbr = np.round(ann['bbox'])
    if( 'dp_masks' in ann.keys()):

```

```

Point_x = np.array(ann[ 'dp_x' ])/ 255. * bbr[2]
Point_y = np.array(ann[ 'dp_y' ])/ 255. * bbr[3]
#
Point_I = np.array(ann[ 'dp_I' ])
Point_U = np.array(ann[ 'dp_U' ])
Point_V = np.array(ann[ 'dp_V' ])
#
x1,y1,x2,y2 = bbr[0],bbr[1],bbr[0]+bbr[2],bbr[1]+bbr[3]
x2 = min( [ x2,I.shape[1] ] ); y2 = min( [ y2,I.shape[0] ] )
#####
Point_x = Point_x + x1 ; Point_y = Point_y + y1

# 根据已读取的标注，得到 iuv 数组
for x, y, i, u, v in zip(Point_x, Point_y, Point_I, Point_U,
Point_V):
    # i 范围 0-24，其中 0 表示无纹理映射，1-24 表示曲面索引；
    # u、v 范围 0-199，表示在纹理子图中的坐标
    y = round(y); x = round(x)
    I_im[y, x] = round(i)
    U_im[y, x] = round(u*200)
    V_im[y, x] = round(v*200)

```

再将转换好的格式传入 TransferTexture 函数中，进行可视化。

```

import os
# 如果路径不对，将 DensePoseData_dir 改为你的绝对路径
# DensePoseData_dir = "D:\projects\DensePose\DensePoseData"
DensePoseData_dir = os.path.abspath("../DensePoseData")
Tex_Atlas = cv2.imread(os.path.join(DensePoseData_dir,
'demo_data/texture_from_SURREAL.png'))[:, :, :-1]/255.
#
TextureIm = np.zeros([24,200,200,3]);
# 将“图集”表示的人体纹理，按照 24 个部分分割
for i in range(4):
    for j in range(6):
        TextureIm[(6*i+j) , :, :, :] = Tex_Atlas[ (200*j):(200*j+200) ,
(200*i):(200*i+200) , : ]

# IUUV 纹理图像、im 原始图像，image 纹理映射后的图像
#####
## Visualize the image with black background, 黑色背景
IUUV =
np.concatenate((I_im[:, :, np.newaxis], U_im[:, :, np.newaxis], V_im[:, :, np.n
ewaxis]), axis =2 ).astype(np.uint8)

```

```
image = TransferTexture(TextureIm,I,IUV)
fig = plt.figure(figsize=[14,14])
plt.imshow(image[:, :, ::-1]); plt.axis('off'); plt.show()
```

结果如下，可以看到纹理已经成功映射（可观察到映射为粉色上衣与灰色裤子）。



由于点较为稀疏，我们再模仿前文可视化 mask 与关键点时 plt.scatter 设笔刷大小为 22 的思路，将关键点直接放大。

```
for ann in anns:
    .....
    for xx, yy, i, u, v in zip(Point_x, Point_y, Point_I, Point_U,
Point_V):
        xx = round(xx); yy = round(yy)
        # 由于关键点较少，此处放大到 10*10
        for x in range(max(xx-10, 0), min(xx+10, I.shape[1]-1)):
            for y in range(max(yy-10, 0), min(yy+10, I.shape[0]-1)):
                # i 范围 0-24，其中 0 表示无纹理映射，1-24 表示曲面索引；
                # u、v 范围 0-199，表示在纹理子图中的坐标
                I_im[y, x] = round(i)
                U_im[y, x] = round(u*200)
                V_im[y, x] = round(v*200)
```

最终得到更直观的纹理映射可视化。

